RAMDE - Engenharia Orientada a Requisitos e Modelos

Mestrado em Engenharia de Sistemas Computacionais Críticos

Lecture 06

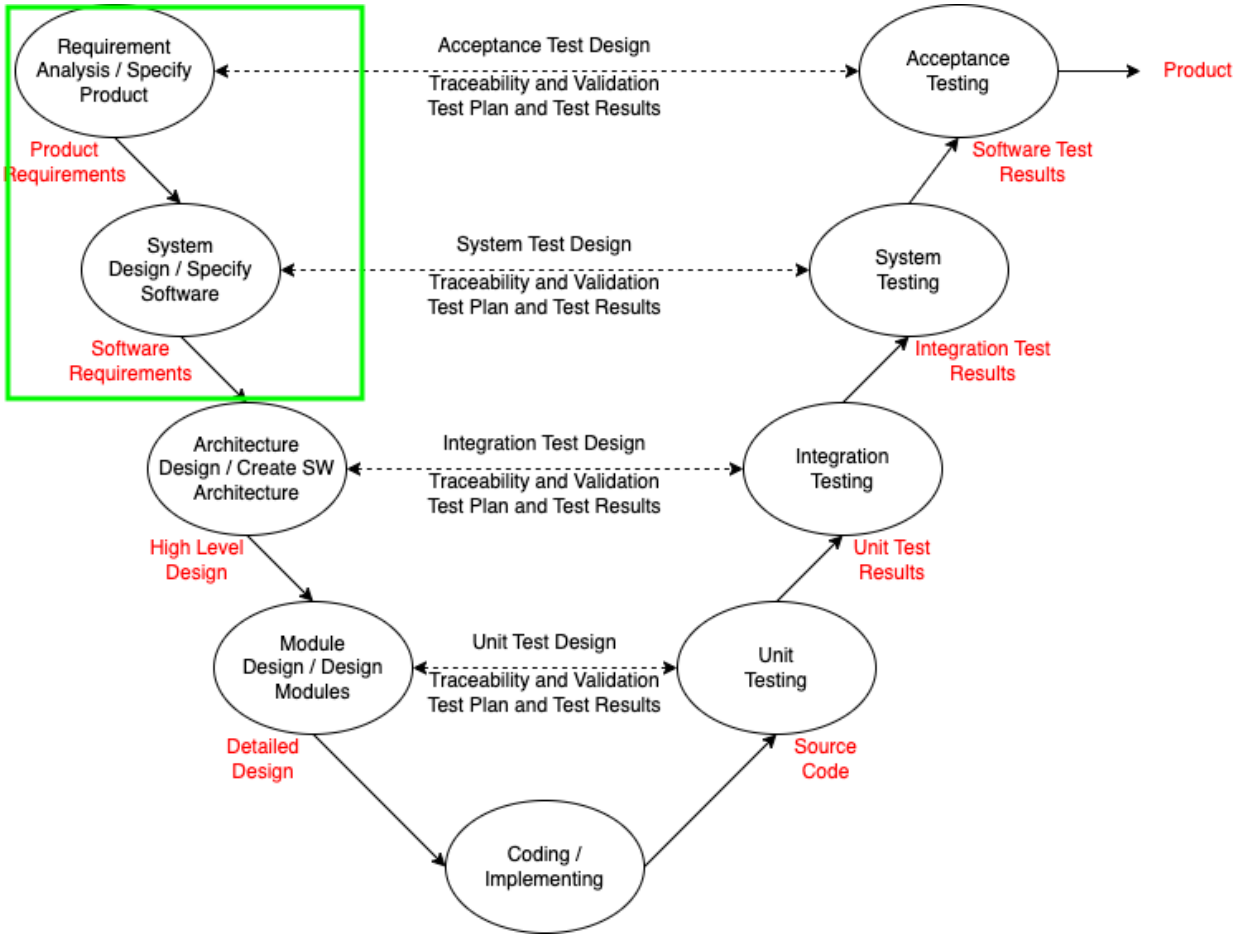
Requirements

Alexandre Bragança atb@isep.ipp.pt

Dep. de Engenharia Informática – ISEP

2026/2027

Requirements



Definitions (1/2)

Requirement[1]

1. A condition or capability needed by a user **to solve a problem or achieve an objective**.
2. A condition or capability that must be met or possessed by a system or system component **to satisfy a contract, standard, specification, or other formally imposed documents**.
3. A documented representation of a condition or capability as in (1) or (2).

Stakeholder

1. A stakeholder of a system is a person or an organization that **has** an (direct or indirect) **influence on the requirements of the system**.

[1] 610.12-1990 - IEEE Standard Glossary of Software Engineering Terminology

Definition (2/2)

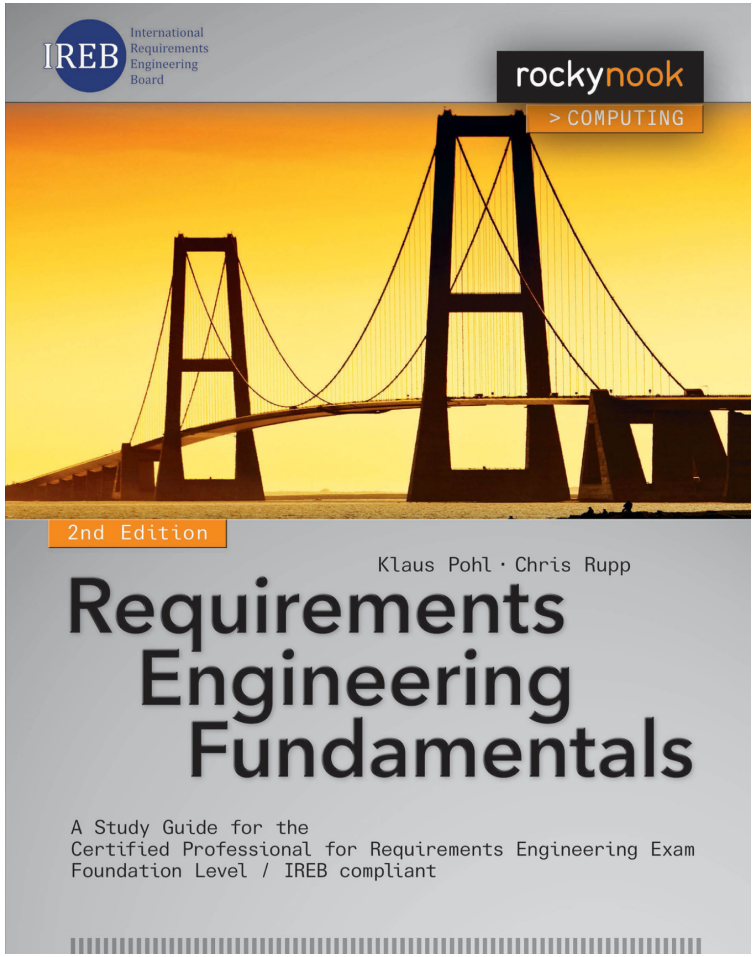
Requirements Engineering

1. Requirements engineering is a systematic and disciplined approach to the specification and management of requirements with the following goals:
   1. **Knowing the relevant requirements**, achieving a consensus among the stakeholders about these requirements, **documenting them** according to given standards, and **managing them systematically**
   2. Understanding and documenting the stakeholders' desires and needs, thus specifying and managing requirements to minimize the risk of delivering a system that does not meet the stakeholders' desires and needs

Four core activities of requirements engineering

- Elicitation
- Documentation
- Validation and negotiation
- Management

The Contents of These Slides are Based On...



"Requirements engineering fundamentals: a study guide for the certified professional for requirements engineering exam-foundation level-IREB compliant", Klaus Pohl, Chris Rupp. Rocky Nook Inc., 2016

www.ierg.org IREB, the International Requirements Engineering Board - a non-profit organization - is the provider of the CPRE (Certified Professional for Requirements Engineering) certification scheme.
The Certified Professional for Requirements Engineering represents a carefully elaborated three level certification concept. It addresses practitioners working in Business Analysis, Requirements Engineering and Testing, who place high demands on the quality of their work. It is IREB's declared goal to foster further education in the field of Requirements Engineering.

Contents

1. Introduction
2. System and Context Boundaries
3. Eliciting Requirements
4. Documenting Requirements
5. Documenting Requirements in Natural Language
6. Model-Based Requirements Documentation
7. Requirements Validation
8. Requirements Management

1- Introduction - Communication Theory

- Language as a medium for requirement communication
  - Requirements must be communicated. In most cases, one uses a rule-driven medium that is accessible to the communication partner—**natural language**.
- Type of communication medium
  - In **verbal communication**, the success of the communication **relies heavily on redundancy** (e.g., language and gestures or language and intonation) **and feedback**.
  - In **written technical communication**, for example, information is transmitted with a **minimum of redundancy and feedback**.
- Language comfort
  - In addition to the problems arising from differing domain vocabularies and different communication media, it can often be observed that information is not adequately transmitted or not transmitted at all
  - This can be traced back to natural transformations that occur during human perception. **These transformational effects are, in particular, focusing and simplification and can impact the communication more or less harshly**.
- Implicit background knowledge
  - Communication—i.e., the language-based expression of knowledge— is necessarily simplifying in nature. The author expects the reader to have some kind of implicit background knowledge.
  - It is the simplifications that arise from language-based knowledge expression that become problematic with regard to requirements, as requirements can become interpretable in different ways.

To be able to fulfill all of her tasks, the requirements engineer needs much more than process knowledge. Many of the capabilities required must be based on practical experience.

- Analytic thinking
- Empathy
- Communication skills
- Conflict resolution skills
- Moderation skills
- Self-confidence
- Persuasiveness

1- Introduction - Requirement Types

Generally, one can distinguish between three types of requirements:[2]

- Functional requirements
  - Define the functionality that the system to be developed offers.
  - Usually, these requirements are divided into **functional requirements, behavioral requirements, and data requirements**
- Quality requirements
  - Define desired qualities of the system to be developed and often influence the system architecture more than functional requirements do.
  - Typically, quality requirements are about the **performance, availability, dependability, scalability, or portability of a system**.
  - Requirements of this type are frequently classified as non-functional requirements.
- Constraints
  - Constraints cannot be influenced by the team members.
  - Requirements of this type can constrain the system itself (e.g., "The system shall be implemented using web services") or the development process ("The system shall be available on the market no later than the second quarter of 2012").
  - In contrast to functional and quality requirements, **constraints are not implemented, they are adhered to** because they merely limit the solution space available during the development process.

[2] A number of different classifications of requirements are used in practice. For example, there are a number of classifications suggested by several standards, e.g., CMMI [SEI 2006] or SPICE [ISO/IEC 15504-5].

The following categories are typical for quality requirements (see [ISO/IEC 25010:2011]):

- Requirements that define the **performance of the system**, in particular response time behavior and resource utilization
- Requirements that define the **security of the system**, in particular with regard to accountability, authenticity, confidentiality, and integrity
- Requirements that define the **reliability of functionalities**, in particular with regard to availability, fault tolerance, and recoverability
- Requirements that define the **usability of a system**, in particular with regard to accessibility, learnability, and ease of use
- Requirements that define the **maintainability of a system**, in particular with regard to reusability, analyzability, changeability, and testability
- Requirements that define the **portability of a system**, in particular with regard to adaptability, installability, and replaceability

2- System and Context Boundaries - System Context

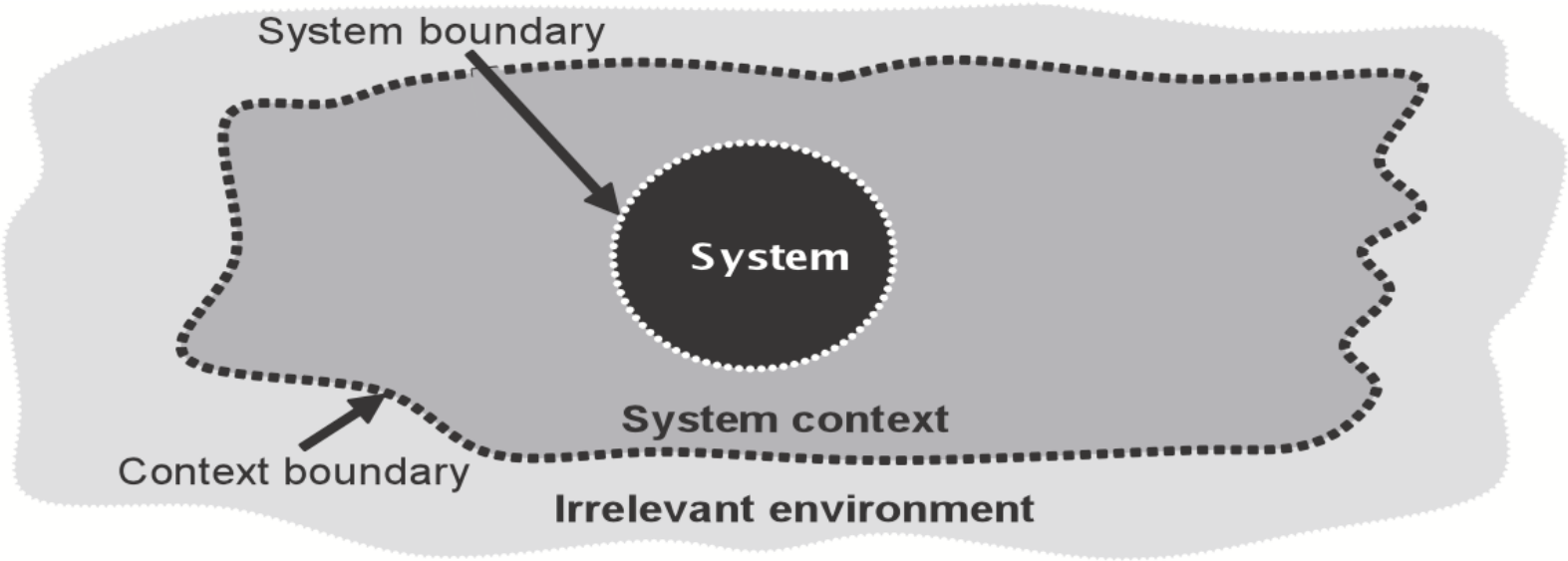
System Context

- The system context is **the part of the system environment that is relevant for the definition as well as the understanding of the requirements of a system to be developed**.
- A requirement is therefore defined for a specific context and can only be interpreted correctly in regard to this specific context

Among others, the following possible aspects of reality influence the context of a system:

- People (stakeholders or groups of stakeholders)
- Systems in operation (other technical systems or hardware)
- Processes (technical or physical processes, business processes)
- Events (technical or physical)
- Documents (e.g., laws, standards, system documentation)

2- System and Context Boundaries - Defining...



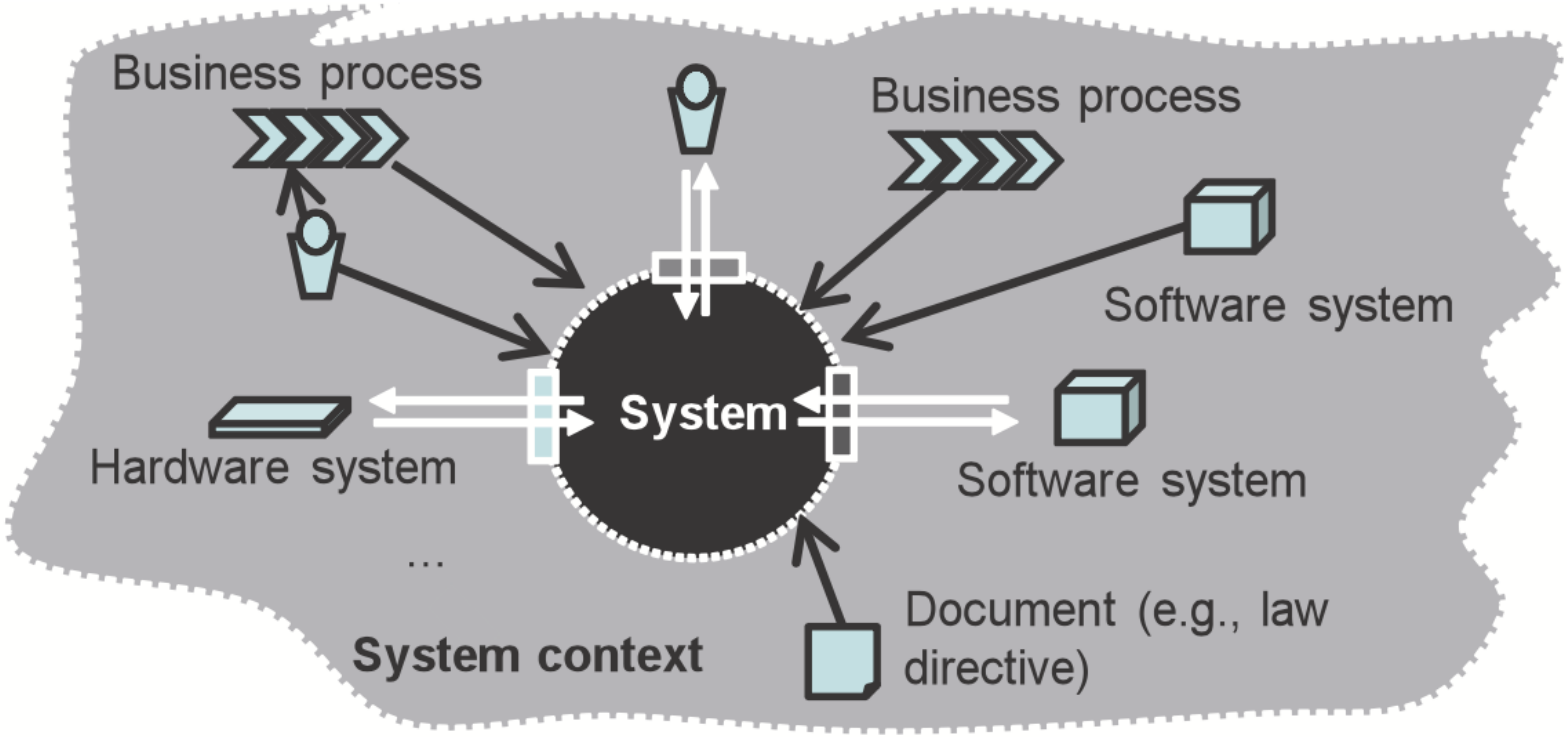
Defining the system boundary

- When defining the system boundary, a decision has to be made: Which aspects pertain to the system to be developed and which aspects belong in the system context?

Defining the context boundary:

- When defining the context boundary, the question to be answered is: Which aspects pertain to the system context (i.e., have a relation to the system to be developed) and which aspects are part of the irrelevant environment?

2- System and Context Boundaries - System Context...



Among other things, **sources and sinks** (see, e.g., [DeMarco 1978]) can be used to identify the interfaces the system has with its environment. Sources provide inputs for the system. Sinks receive outputs from the system. Possible sources and sinks of a system are as follows:

- (Groups of) stakeholders
- Existing systems (both technical and nontechnical systems)

2- System and Context Boundaries - Document...

In order to document the system context of a system as thoroughly as possible, typically several documentation forms are used.

- **UML and use case diagrams** can be used.
  - In use case diagrams, actors (such as people or other systems) in the system environment and their usage relationships to the system are modeled.
  - To model the system context, UML class diagrams may also be used
- **Data flow diagrams** [DeMarco 1978] are other option
  - When the context is modeled with data flow diagrams, sources and sinks in the environment of the system that represent the source or destination of data flows (or flows of material, energy, money, etc.) are modeled

3- Eliciting Requirements - Requirements Sources

The basis for requirements elicitation **is the knowledge** that has been gained during requirements engineering **about the system context** of the system to be developed, **which comprises the requirements sources that are to be analyzed and queried**.

There are three different **kinds of requirements sources**:

- Stakeholders
- Documents
- Systems in operation

A requirements specification is a systematically represented collection of requirements, typically for a system or component, that satisfies given criteria.

- Requirements are the basis of the system development
- Requirements have a legal relevance
- Another argument for a good documentation, supportive of the project, is that stakeholders almost never share the same understanding of a subject matter.

Documentation Technique

- A documentation technique is any kind of more or less formal depiction that eases communication between stakeholders and increases the quality of the documented requirements.
- In principle, any kind of documentation technique can be used to document the requirements, let it be:
  - **natural language**-based documentation by means of prose,
  - more **structured natural language**-based text,
  - or more **formal techniques** such as state diagrams.

Requirements for a system can be documented in three different perspectives.

- **Data perspective**
  - A static-structural perspective on the requirements of the system is adopted.
  - For example, the structure of input and output data as well as static-structural aspects of usage and dependency relations of the system and the system context can be documented
- **Functional perspective**
  - Documents which information (data) is received from the system context and manipulated by the system or one of its functions.
  - This perspective also documents which data flows back into the system context.
  - The order in which functions processing the input data are executed is also documented.
- **Behavioral perspective**
  - Information about the system and how it is embedded into the system context is documented in a state-oriented manner.
  - This is done by documenting the reactions of the system upon events in the system context, the conditions that warrant a state transition, and the effects that the system shall have on its environment

4- Documenting Requirements Requirements - Natural Language

- Natural language, particularly prose, is **the most commonly applied documentation form for requirements in practice**.
- In contrast to other documentation forms, prose has a striking advantage: No stakeholder has to learn a new notation.
- Natural-language-based documentation is well suited to document requirements in any of the three perspectives.
- However, **natural language can allow requirements to be ambiguous**, and requirements of different types and perspectives are in danger of being unintentionally mixed up during documentation.

- **Conceptual models offer a decreased degree of ambiguity than natural language due to their higher degree of formality**.
- Most important diagrams used:
  - **Use case diagram** - quick overview of the functionalities of the specified system
  - **Class diagram** - among other things, class diagrams are used in requirements engineering to document requirements with regard to the **static structure of data**, to document static-structural dependencies between the system and the system context, or to document complex domain terms in a structured manner
  - **Activity diagram** - Using activity diagrams, business processes, or sequence-oriented dependencies of the system in regard to processes within the system context can be documented. Activity diagrams are also well suited to model the sequential character of use cases or to model a **detailed specification of the interaction of functions that process data**
  - **State diagram** - State diagrams are used in requirements engineering to document **event-driven behavior of a system**. The focus of this type of model is on the individual states the system can be in, events and their respective conditions that trigger a state transition, and effects of the system in its environment

- Requirements documents contain a magnitude of different information.
- These must be well structured for the reader. In order to do that, one can make use of **standardized document structures** or individually define a **custom document structure**.

Standardized Document Structures

- The Rational Unified Process (RUP) is usually used for software systems that are developed using object-oriented methods
- The ISO/IEC/IEEE standard 29148:2011 [ISO/IEC/IEEE 29148:2011] contains an outline designed for the documentation of software requirements (software requirements specification)
- The V-Model of the German Federal Ministry of the Interior (BMI) defines different structures, depending on the creator of the requirements document

To become a basis for the subsequent processes, the requirements document must meet certain quality criteria. According to the ISO/IEC/IEEE standard 29148:2011, **a requirements document shall be complete and consistent**. Overall, a requirements document shall fulfill the following quality criteria:

- **Unambiguity and consistency**
- **Clear structure**
- **Modifiability and extendibility**
- **Completeness**
- **Traceability**
- **Verifiability**

4- Documenting Requirements Reqs - Enhance Readability

Along with quality criteria for requirements, there are two fundamental rules that enhance the readability of requirements:

- **Short sentences and short paragraphs** - As human short-term memory is very limited, circumstances that belong together should be described in no more than seven sentences.
- **Formulate only one requirement per sentence** - Formulate requirements using active voice and use only one process verb. Long, complicated interlaced sentences must be avoided.

4- Documenting Requirements Reqs - Glossary

By defining the meaning of terms, you can increase the understandability of requirements considerably. Misunderstandings and different interpretations of terms that might lead to conflicts can be avoided from the beginning.

A glossary is a collection of term definitions and contains the following elements:

- **Context-specific technical terms**
- **Abbreviations and acronyms**
- **Everyday concepts that have a special meaning in the given contex**
- **Synonyms, i.e., different terms with the same meaning**
- **Homonyms, i.e., identical terms with different meanings**

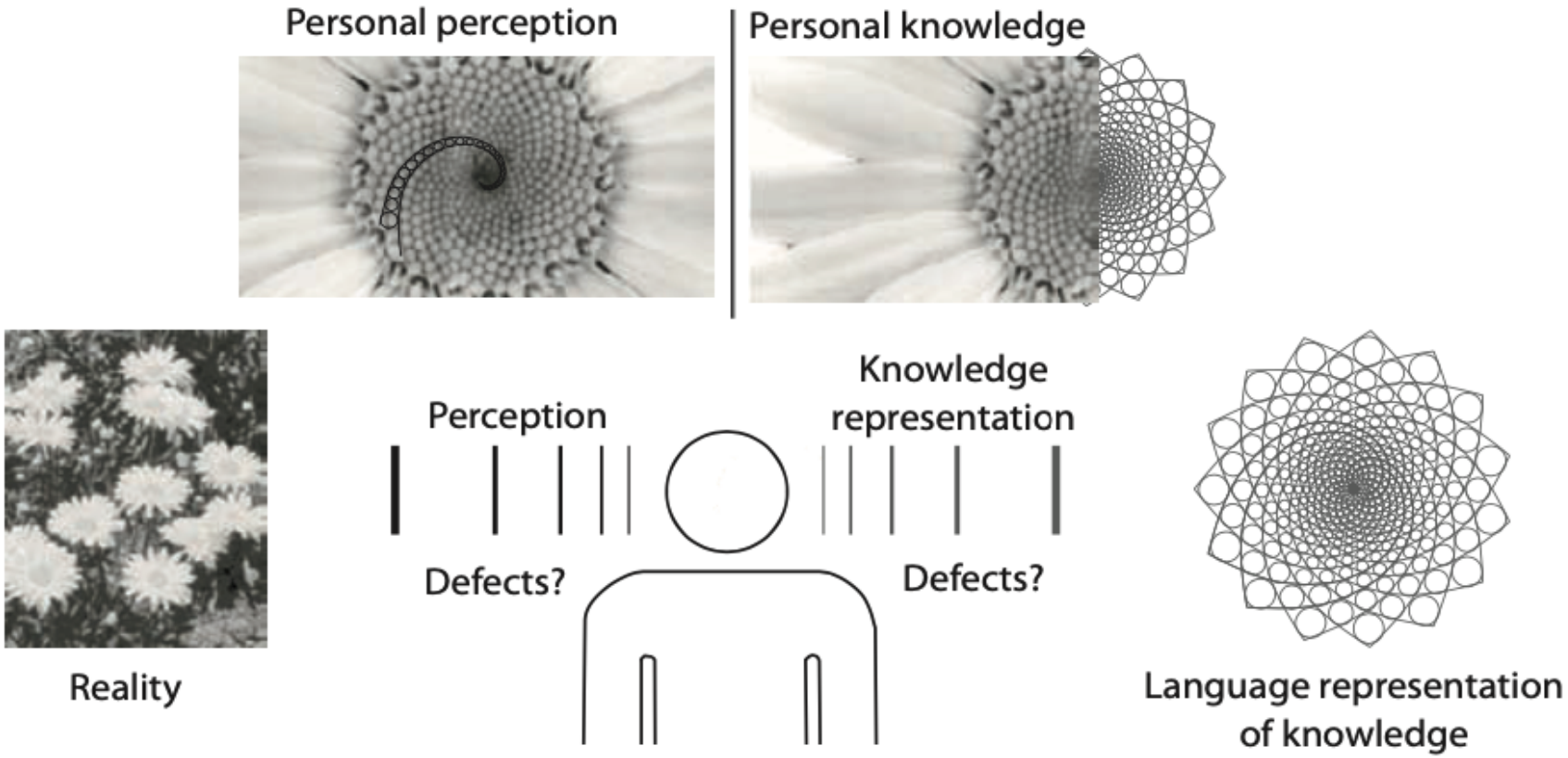
5- Documenting Requirements in Natural Language

- Elicited requirements for the system to be developed are **frequently documented using natural language**
- Natural language has the advantage that it (allegedly) does not require preparation time in order to be read and understood by stakeholders
- In addition, natural language is universal in the sense that it can be used to describe any circumstances.
- However, **there are some problems associated with the use of natural language for requirements documentation...**

Transformational Effects of Natural Language

As **natural language is inherently ambiguous** and statements in natural language can often be interpreted in multiple ways, it is necessary to place special emphasis on potential ambiguities in such statements to satisfy the criterion of unambiguousness.

Transformation Effects in Perception and Representation of Knowledge:



Transformational Effects of Natural Language

The fact that **transformational effects** adhere to certain **rules** can be exploited by the requirements engineer to **elicit the deep structure** (i.e., what the author of a requirement really meant) from its **surface structure** (i.e., the requirements).

The following list includes the **five transformational processes** that are most relevant for requirements engineering:

1. Nominalization
2. Nouns without reference index
3. Universal quantifiers
4. Incompletely specified conditions
5. Incompletely specified process verbs

1- Nominalization

By means of nominalization, a (sometimes long-lasting) process is converted into a (singular) event. **All information necessary to accurately describe the process is thereby lost**. The process word transmit turns into the noun transmission. Other typical examples of nominalization are the terms input, booking, and acceptance.

Example of Nominalization

- "In case of a system crash, a restart of the system shall be performed."
- The terms **system crash** and **restart** each describe a process that ought to be analyzed more precisely.
- It is therefore not necessary to avoid nominalizations, but they **should only be used if the underlying process is completely defined**.

2- Nouns without Reference Index

As with process verbs, **nouns are frequently incompletely specified**. Linguists call this a missing or inadequate index of reference. Examples of terms that contain incompletely specified nouns are **the user, the controller, the system, the message, the data, or the function**.

Example of Nouns without reference indices

- "The data shall be displayed to the user on the terminal."
- The following questions arise: What data exactly? Which user exactly? Which terminal exactly?

If this information is amended, the requirement might thus read as follows:

- "The system shall display the billing data to the registered user on the terminal she is logged in to."

3- Universal Quantifiers

Universal quantifiers specify **amounts or frequencies**. They group a set of objects and make a statement about the behavior of this set. When using universal quantifiers, **there is the risk that the specified behavior or property does not apply to all objects within the specified set**.

It must be verified whether the specified behavior really applies to all objects summarized through the quantifiers. Universal quantifiers can be easily identified through trigger words such as **never, always, no, none, every, all, some, or nothing**.
Example of Universal Quantifiers

- "The system shall show all data sets in every submenu."
- In this case, the following question must be asked: Really in every submenu? Really all data sets?

4- Incompletely Specified Conditions

Incompletely specified conditions are another indicator of a potential loss of information. Requirements that contain conditions specify the behavior that must occur when the condition is met. In addition, they must specify **what behavior must occur if the condition is not met (the part that is often missing)**.

Especially in complex conditional structures, decision tables can be invaluable tools to find unspecified variants of conditions or actions. Trigger words are, for instance, **if ... then, in case, whether, and depending on**.
Example of Incompletely Specified Conditions

- "The restaurant system shall offer all beverages to a registered guest over the age of 20 years."
- At least one aspect remains unspecified in the example above: Which beverages shall be offered to a guest that is 20 years or younger?

Clarifying this question may lead to extending the requirement as follows:

- "The restaurant system shall offer
- All alcohol-free beverages to any registered user younger than 21 years
- All beverages including all alcoholic beverages to any user over the age of 20"

5- Incompletely Specified Process Verbs

Some **process verbs require more than one noun to be considered completely specified**. The **verb transmit**, for instance, requires at least three supplements to be considered complete: **what is being transmitted, from where it is being transmitted, and to where it is being transmitted**.

The use of incompletely specified process words **can mostly be avoided or kept to a minimum if requirements are formulated using the active voice** rather than the passive voice.
Example of Requirement using the passive voice

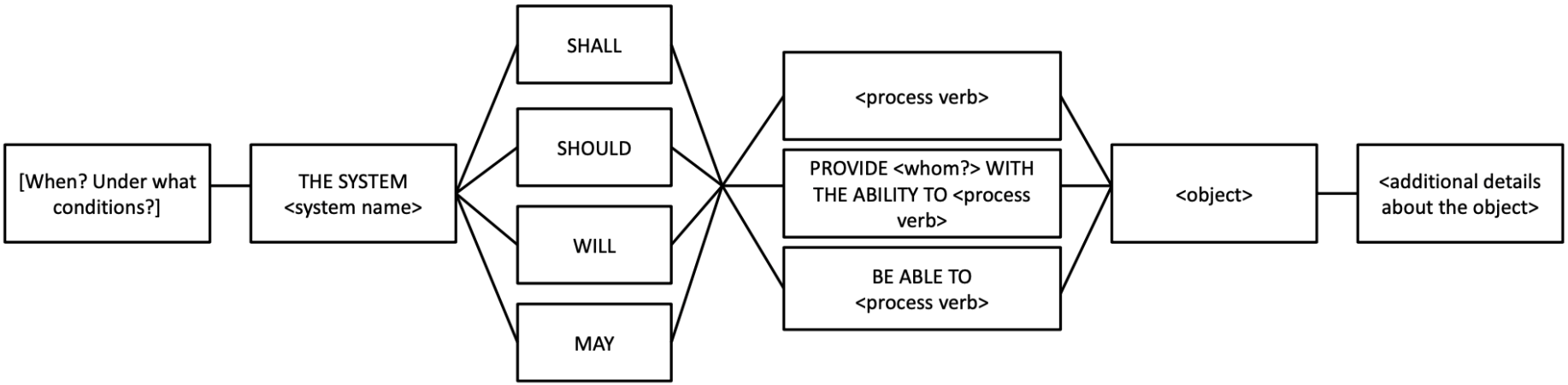
- "To log a user in, the login data is entered."
- In this requirement using passive voice, it is unclear who enters the login data. It is also unclear where and how this is done.

If this requirement is reformulated using the active voice, at least the agent or person responsible must be included.

- "The system must allow the user to enter his user name and password using the keyboard of the terminal."

Requirement Construction Using Templates

- Requirements templates provide a simple and easily understandable approach to reduce language effects when documenting requirements.
- Templates support the author in achieving high quality and syntactic unambiguousness in optimal time and at low costs.
- In some sort similar to how "user stories" work (see `https://www.atlassian.com/agile/project-management/user-stories`) in agile approaches



5- Documenting Requirements in Natural Language - Practical Examples

EARS (Easy Approach to Requirements Syntax)

- Is an effective method of expressing requirements in "natural language"
- Tutorial: `https://www.iaria.org/conferences2013/filesICCGI13/ICCGI_2013_Tutorial_Terzakis.pdf`
- Paper: `https://ieeexplore.ieee.org/document/5328509`
- Site: `https://alistairmavin.com/ears/`

6- Model-Based requirements Documentation

During model-based documentation of requirements in requirements engineering, three types of requirements are documented independently and used in conjunction:

- **Goals** describe intentions of stakeholders or groups of stakeholders. Goals can potentially conflict with one another.
- **Use cases and scenarios** document exemplary sequences of system usage. Scenarios are grouped together in use cases.
- **System requirements** (generally referred to as requirements) describe detailed functions and qualities that the system to be developed shall implement or possess. In addition, system requirements provide complete and precise information to serve as input for subsequent development steps.

In practice, requirements are often documented using natural language. However, **it can be observed that requirements are increasingly often documented using models**. Requirement models are used in addition to natural language requirements documentation and partly replace requirements that would have been documented using natural language.

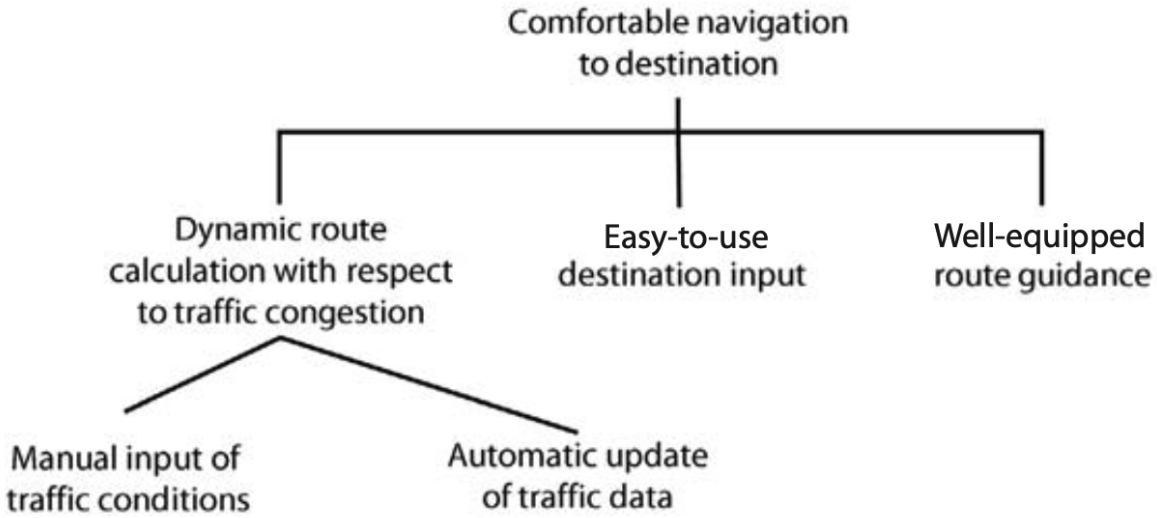
Requirements Models

- Conceptual models that document the requirements of a system are called requirements models
- The Unified Modeling Language (UML) is frequently used to construct requirements models (See `https://www.omg.org/spec/UML/`)
- UML has developed into the quasi-standard for model-based construction of software systems

Goal Models

- Goals can be documented using natural language (e.g., by means of predesigned templates) or using goal models.

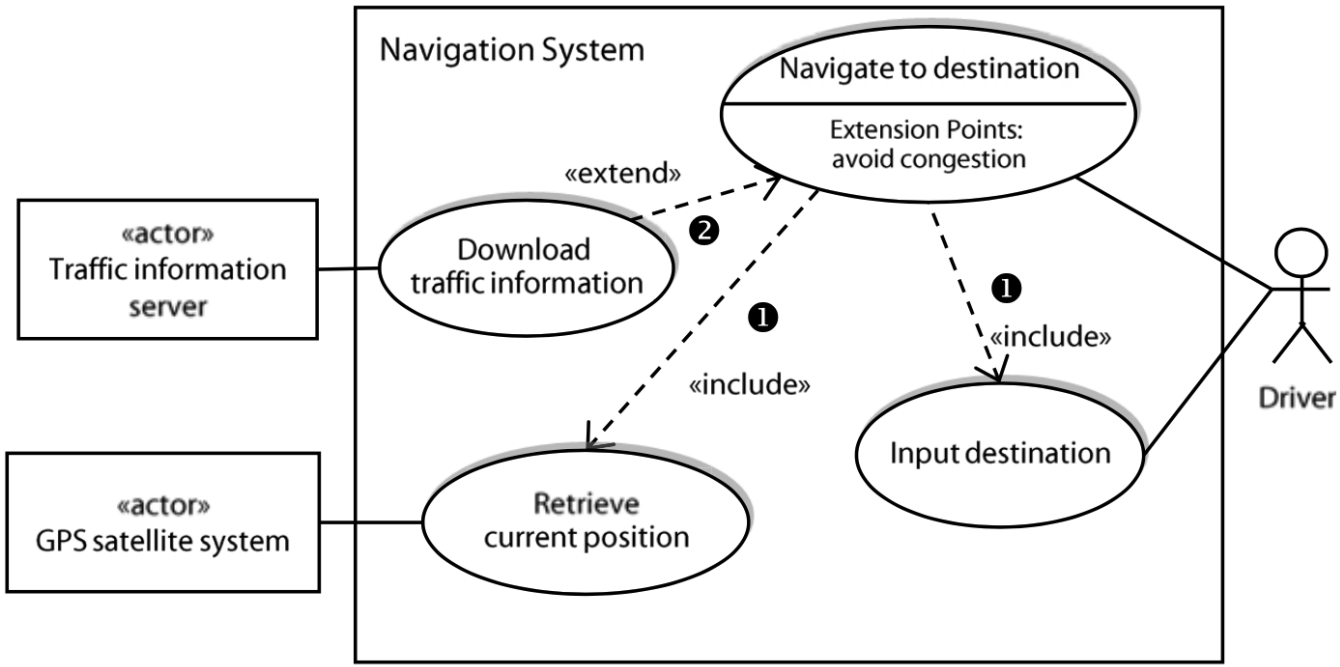
Example of a Goal Model (AND/OR tree)



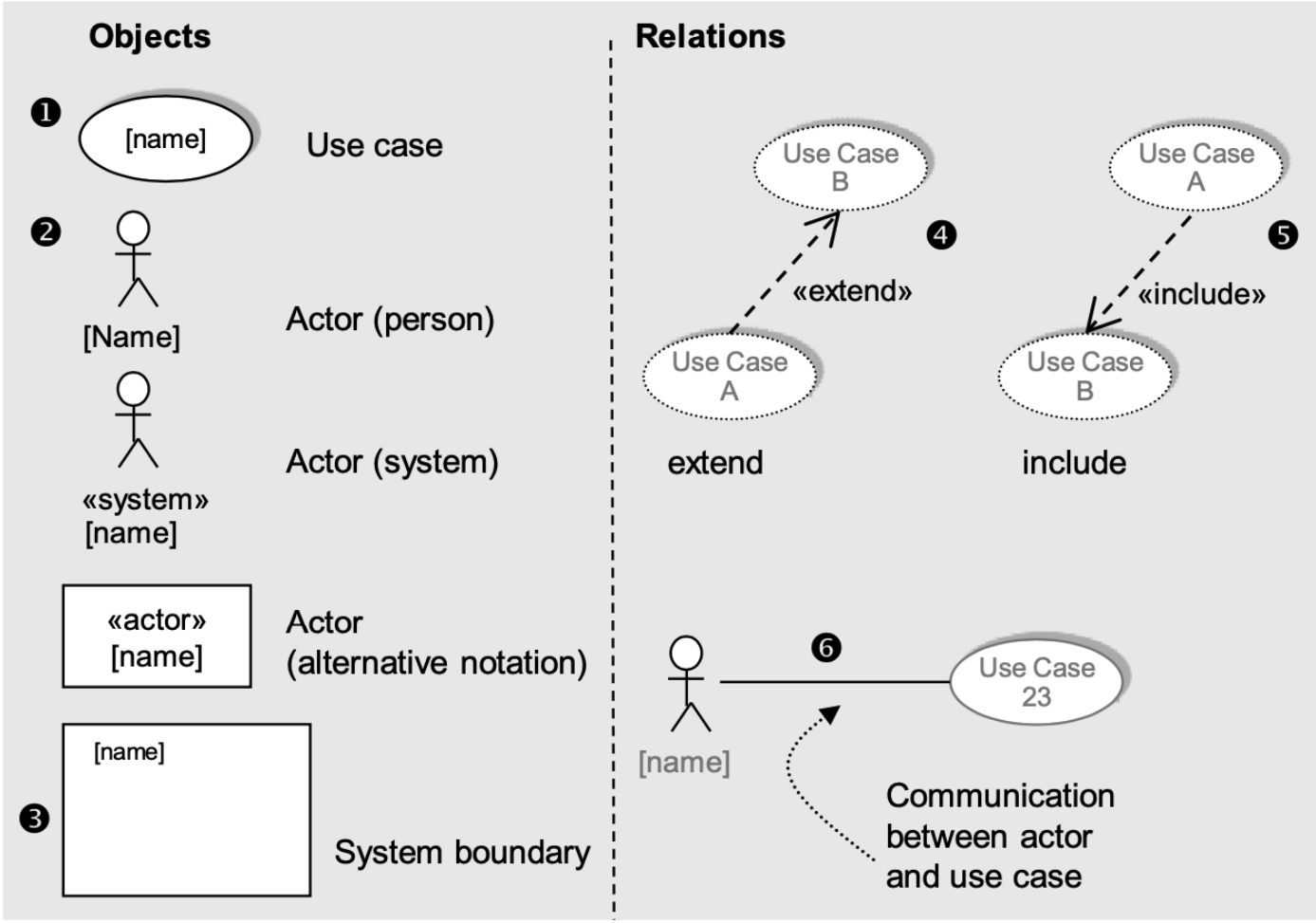
Use Cases

- Use cases were first proposed in [Jacobson et al. 1992] as a method to document the functionalities of a planned or existing system on the basis of simple models
- They are a part of UML

Example of a Use Case diagram



Essential modeling elements of use case diagrams



Use Case Specifications

- With the exception of a use case's name and its relationships, use cases diagrams do not document any information about the individual use cases such as the systematic interaction between a use case and an actor

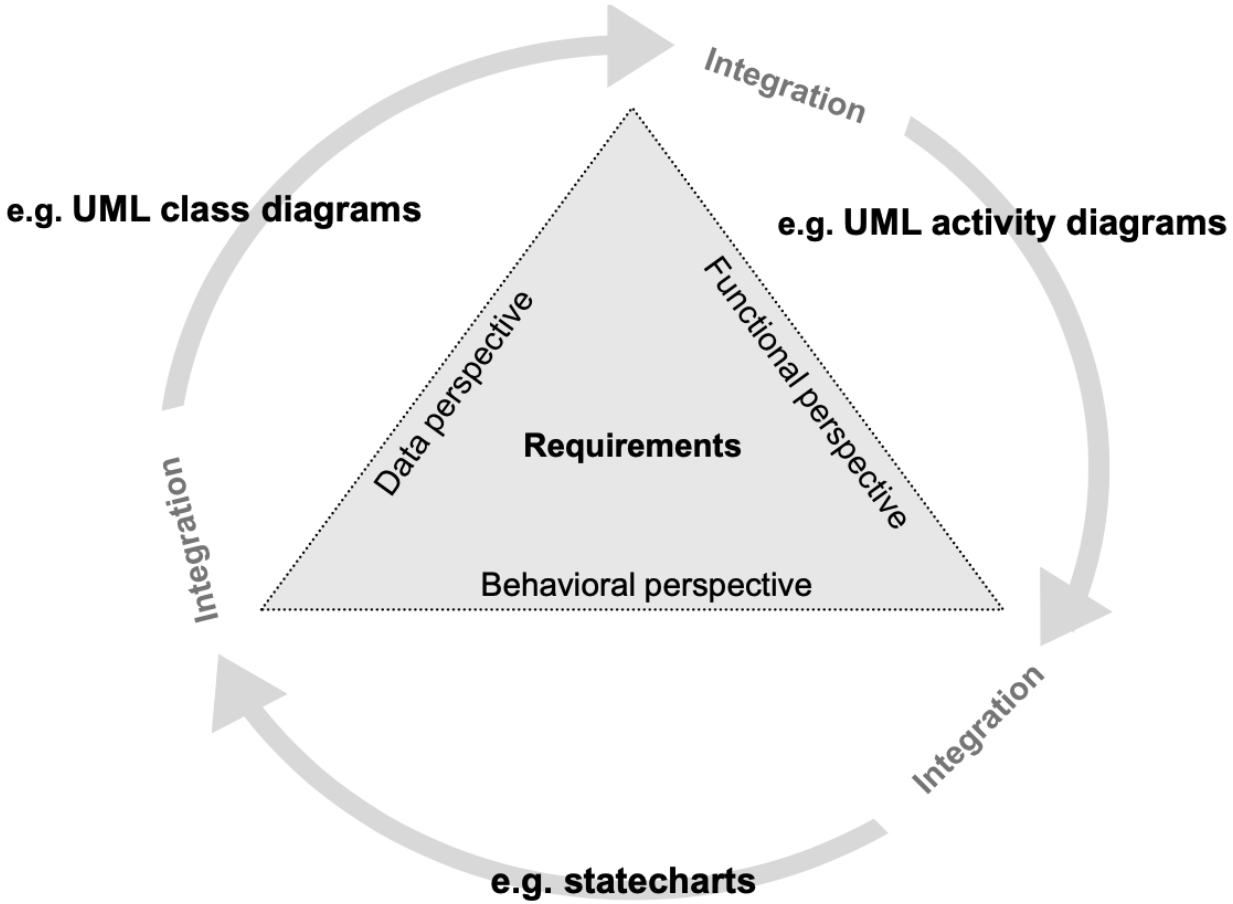
- This information is documented textually by means of adequate templates in conjunction with use case diagrams, in **Use Case Specifications**.

Template for Textual Use Case Documentation		
Nr.	**Section**	**Content / Explanation**
1	Designation	Unique designation of the use case.
2	Name	Unique name of the use case.
3	Authors	Names of the authors that were involved in this use case description.
4	Priority	Importance of the use case according to the applied prioritization technique.
5	Criticality	Criticality of the use case, e.g., with respect to how much damage a failure of the use case may cause.
6	Source	Designation of the source from which the use case was elicited ([stakeholder | document | system]).
7	Person responsible	The stakeholder who is responsible for this use case.
8	Description	Brief description of the use case.
9	Trigger event	Name of the event that triggers this use case.
10	Actors	List of all actors that are involved in this use case.
11	Pre-conditions	List with all necessary constraints that must be met before the use case can begin execution.
12	Post-conditions	List of all states the system can be in immediately after the execution of the main scenario.
13	Result	Description of the results that are produced during use case execution.
14	Main scenario	Description of the main scenario of the use case.
15	Alternative scenarios	Description of the alternative scenarios of the use case or list of the trigger events of alternative scenarios. Often, post-conditions different than those described in (12) may hold.
16	Exception scenarios	Description of the exception scenarios of the use case or list of the trigger events of exception scenarios. Often, post-conditions different than those described in (12) may hold.
17	Qualities	Cross references to quality requirements.

Three Perspectives on the Requirements

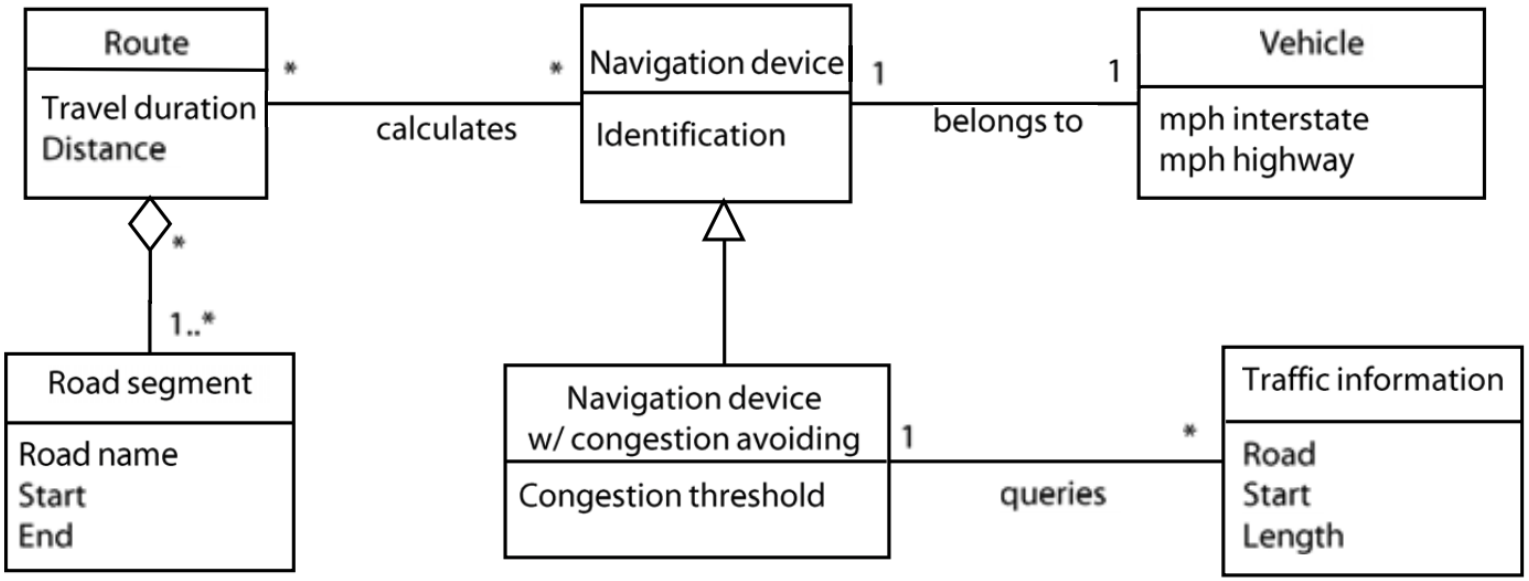
- **Data perspective:** In this perspective, the structures of input and output data as well as static-structural aspects of the usage and dependency relationships of the system in the system context are documented.
- **Functional perspective:** This perspective documents which information of the system context is being manipulated by the system to be developed and which data is being transmitted to the system context by the system.
- **Behavioral perspective:** The embedding of the system in the system context is documented on the basis of states in this perspective. This is done, for instance, by documenting the reaction of the system to events within the system context, documenting the conditions that trigger a state change, or documenting the effects that the system has on its environment.

Three Perspectives on the Requirements



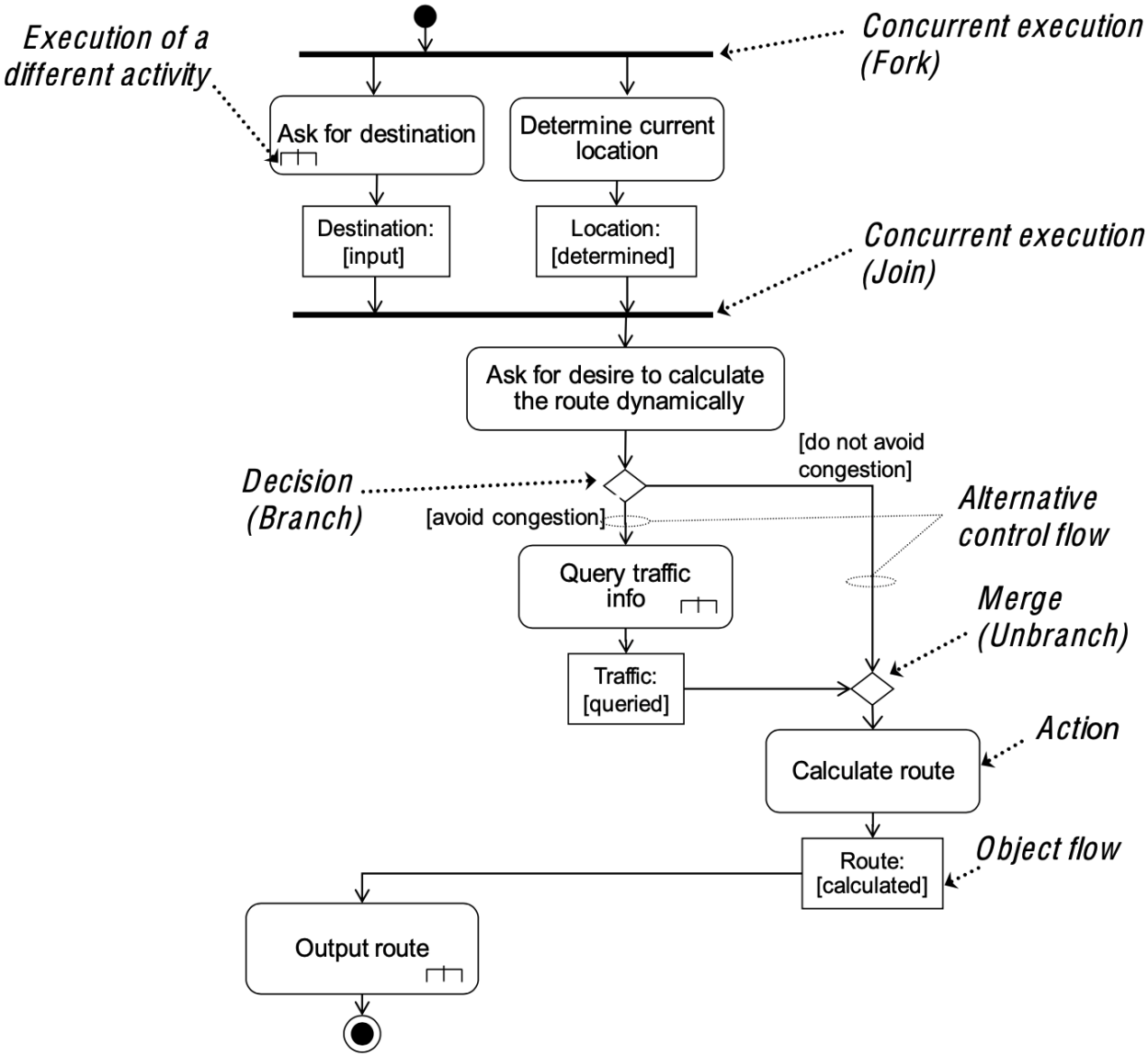
Requirements Modeling in The Data Perspective

- UML Class Diagrams



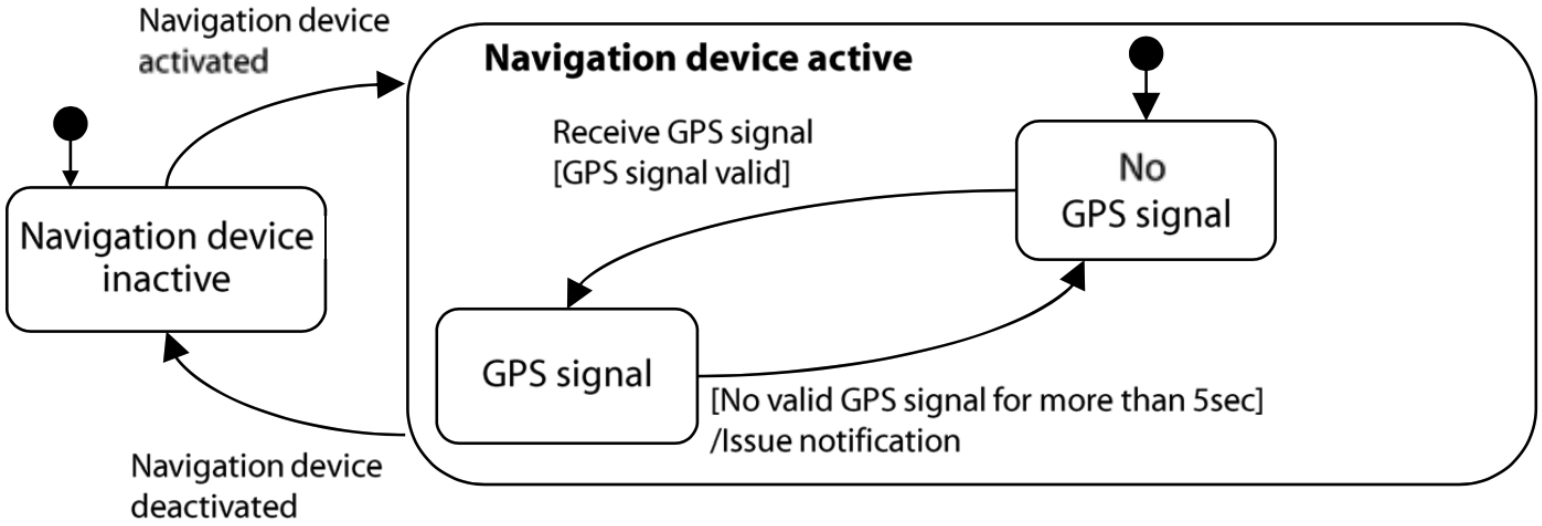
Requirements Modeling in The Functional Perspective

- UML Activity Diagrams



Requirements Modeling in The Behavioral Perspective (1/2)

- Statecharts or State Diagrams



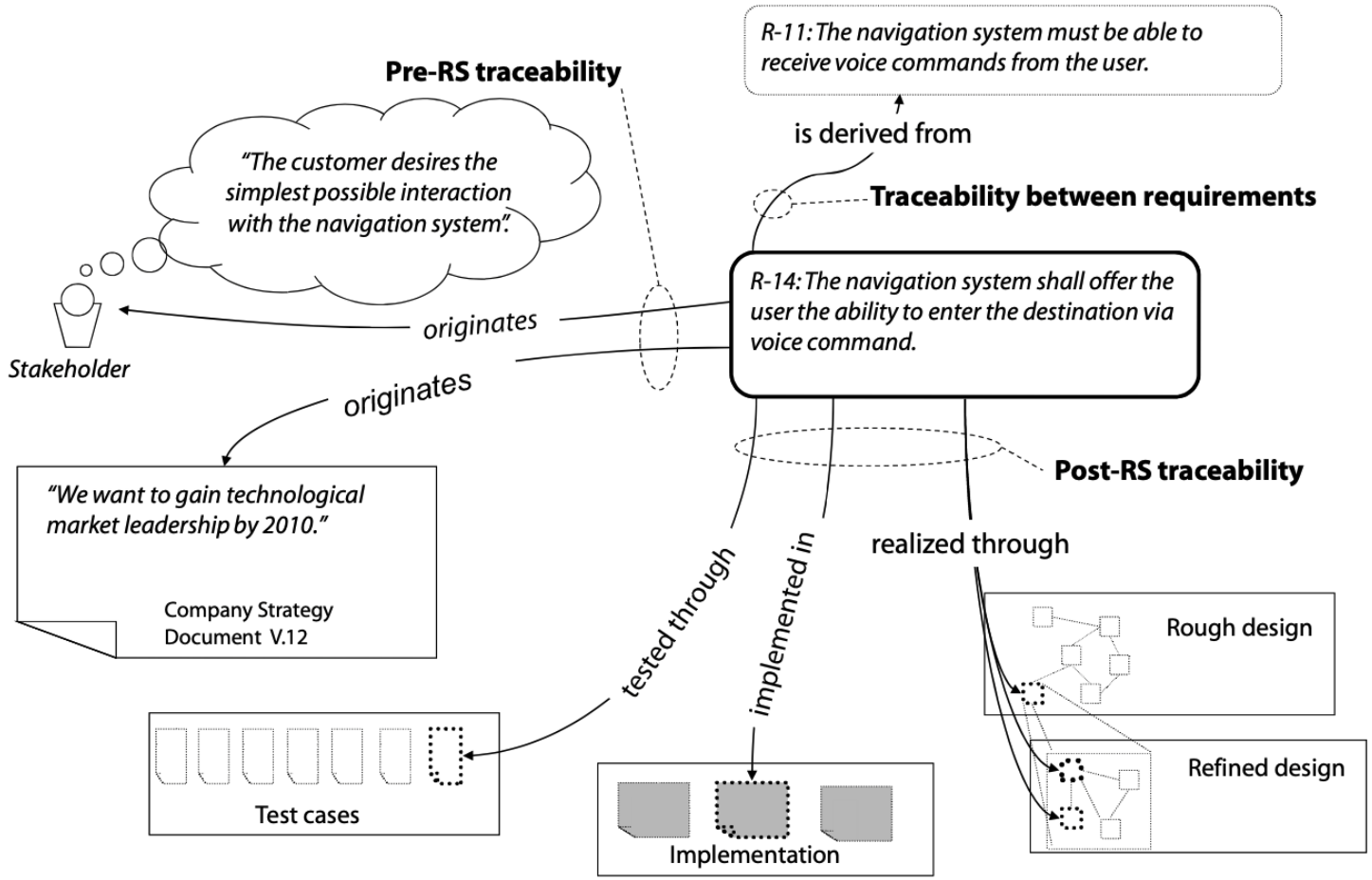
7- Requirements Validation

- During **requirements validation**, the decision of whether a **requirement possesses the necessary level of quality** is made and whether the requirement can be approved to be used for further development activities (such as design, implementation, and testing).
- This decision should be made on the **basis of predefined acceptance criteria**.
- **The goal of requirements validation is therefore to discover errors in the documented requirements**. Typical examples of errors in requirements are **ambiguity, incompleteness, and contradictions**

Traceability of Requirements

An important aspect of requirements management is ensuring the traceability of requirements. **The traceability of a requirement is the ability to trace the requirements over the course of the entire life cycle of the system.**

Example of Types of Requirements Traceability



Advantages of Traceable Requirements

- **Verifiability** Allows verifying whether a requirement has been implemented in the system
- **Impact Analysis** Allows for the analysis of effects during change management. For example, traceability of requirements allows identifying the requirements artifacts that must be changed when their underlying requirements undergo a change
- **Reuse** Allows for the reuse of requirements artifacts in other projects.
- **Accountability** Allows for retroactive assignment of development efforts to a requirement. After the requirement is implemented, for example, all partial efforts for the associated development artifact can be summed up and associated with the requirement.
- **Maintenance** Allows for simplified system maintenance. For example, the cause and effect of failures can be identified, the system components that are affected by the failure can be determined, and the effort for removing the underlying error can be estimated.

Next...

Next... Requirements Modeling...